

PROJET FIL ROUGE

e-Sénégal portail de services administratifs numériques

Cahier de projet technique Laravel monolithique fondé sur une démarche incrémentale

Paramètre	Cadre proposé
Niveau	Intermédiaire
Prérequis	PHP POO, SOLID, SQL, Git, notions Docker et sessions HTTP
Modalité	Travail individuel avec revues de jalons
Charge indicative	36 à 42 heures pour le socle source ; prévoir un temps complémentaire pour workflow, bilinguisme, thèmes et contrôles d'accès
Socle	Laravel monolithique, PHP 8.3 ou 8.4, Blade, MySQL, Composer, GitLab CI/CD
Livraison	15 versions intermédiaires v0.0.0 à v0.14.0 puis v1.0.0

Principe directeur Chaque incrément produit une application exécutable, une version Git identifiable, une évolution maîtrisée de l'environnement, une preuve de pipeline et un DevLog. Les notions apparaissent lorsqu'un problème concret les rend nécessaires.

Compétence finale visée

À l'issue du projet, l'étudiant doit pouvoir concevoir, tester, conteneuriser et livrer un portail administratif Laravel, expliquer ses choix d'architecture, protéger les dossiers des citoyens et distinguer le déploiement du code du déploiement d'une image immuable.

A Description du projet

1 Contexte

Le citoyen consulte les démarches, prépare les pièces, dépose un dossier et suit son traitement. L'agent instruit les dossiers de son administration, demande des compléments et rend une décision traçable.

e-Sénégal est ici un portail pédagogique full Laravel avec Blade et MySQL. Il centralise catalogue, dépôt et suivi, puis progresse vers deux modes de livraison automatisée. Les comptes et procédures de démonstration sont fictifs.

2 Objectifs pédagogiques

- Comprendre le cycle d'une requête Laravel depuis la route jusqu'à la réponse.
- Modéliser les données avec migrations, Eloquent, relations, factories et seeders.
- Valider et transformer les données HTTP avant leur utilisation.
- Séparer orchestration HTTP, règles métier, accès aux données et présentation.
- Utiliser l'injection de dépendances et le conteneur de services Laravel.
- Composer les requêtes Eloquent avec des scopes et maîtriser les effets de bord avec Observers, Events et Listeners.
- Écrire des tests unitaires, fonctionnels et d'intégration pour les permissions, le workflow et les pièces.
- Traiter les décisions concurrentes et garantir la cohérence entre statut et historique.
- Construire une image Docker reproductible et exécuter l'application avec Compose.
- Mettre en place une chaîne GitLab de qualité, publication et déploiement.
- Documenter une progression technique par versions, tags, releases et DevLogs.

3 Socle technique

Besoin	Choix
Framework	Laravel 13 avec PHP 8.3 ou 8.4 ; version exacte figée au jalon 1
Persistance	MySQL 8.4 LTS, InnoDB et Eloquent
Interface	Blade, CSS léger et JavaScript limité aux préférences et interactions
Validation	Form Requests, Policies et invariants dans Services/Actions
Tests	Pest ou PHPUnit, un seul choix justifié ; Unit, Feature, Integration
Qualité	Laravel Pint et Larastan ou PHPStan compatibles avec la version retenue
Conteneurs	Docker progressif puis Dockerfile multi stage et Compose
Forge et CI	GitLab Issues, Merge Requests, Registry, Environments, Releases et CI/CD

Version de Laravel La version exacte est figée dans composer.lock. Toute montée de version en cours de projet est une décision documentée, testée et livrée dans un incrément dédié.

4 Fonctionnalités attendues

Gestion des administrations et du catalogue

- Consulter administrations, services et démarches actives, avec conditions, pièces et délai.
- L'administrateur crée et modifie le catalogue.
- Désactiver sans supprimer les dossiers ni leur historique.
- Paginer et filtrer simplement par administration ou service.

Administration, ServicePublic et Demarche modèle détaillé section 11.

Le catalogue est disponible en français et Wolof.

Gestion des pièces justificatives

- Définir les pièces attendues ; joindre PDF, JPEG ou PNG à un dossier privé.
- Remplacer ou retirer une pièce seulement en BROUILLON ou COMPLEMENT_REQUIS.
- Conserver les anciennes versions ; métadonnées MySQL et fichiers privés MinIO au jalon 12.
- Utiliser d'abord le disque local privé, puis migrer vers MinIO.

PieceJustificative modèle et versionnement détaillés section 11.

Formats PDF, JPEG, PNG ; 5 Mio maximum par fichier et dix pièces courantes par dossier.

Gestion des dossiers et du suivi

- Le citoyen voit ses dossiers ; l'agent voit ceux soumis à son administration.
- Consulter pièces et historique ; demander un complément puis décider.
- Créer, renseigner et soumettre un brouillon.
- Annuler selon la matrice sans supprimer les données.

Dossier modèle détaillé section 11 ; transitions section 12.

Statuts BROUILLON, SOUMIS, EN_ETUDE, COMPLEMENT_REQUIS, VALIDE, REJETE, ANNULE.

5 Règles métier

1. Le citoyen est actif, authentifié et propriétaire ; son identifiant vient de la session.
2. La démarche, son service et son administration sont actifs lors de la création et de la première soumission.
3. L'objet a de 5 à 255 caractères ; les informations au plus 5000.
4. Un brouillon peut être incomplet ; la soumission vérifie toutes les pièces obligatoires.
5. L'agent actif relève de l'administration du dossier et s'affecte celui-ci à la prise en étude.
6. Une transition respecte la matrice, le rôle, l'affectation et la version attendue.
7. Un complément ou rejet exige un motif de cinq à deux mille caractères.
8. Un état terminal est en lecture seule ; aucune annulation ou décision ne supprime les données.
9. Statut, version et historique sont atomiques ; deux décisions concurrentes ne peuvent pas réussir.

Cohérence du workflow Toute modification de statut passe par la même action transactionnelle. L'ancien statut est relu sous verrou, la version attendue est vérifiée, puis le nouvel état et l'historique sont écrits ensemble.

6 Exigences non fonctionnelles

- Afficher des erreurs traduites ; conserver les champs non sensibles ; resélectionner les fichiers.
- Aucun secret ne figure dans le dépôt, l'image ou les journaux.
- Stocker les dates en UTC et les afficher dans Africa/Dakar ; conserver les codes de statut indépendamment de la langue.
- Deux transitions concurrentes d'un même dossier ne produisent jamais deux décisions contradictoires.
- Les pièces sont privées ; autoriser l'accès avant toute URL temporaire et invalider le cache après mutation.

7 Contraintes architecturales

- Les contrôleurs restent minces ; aucune matrice de statuts ou règle de complétude dans le contrôleur.
- Les Form Requests assurent la validation de forme et l'autorisation HTTP.
- Les règles métier sont regroupées dans des services ou actions applicatives.
- Les dépendances sont reçues par constructeur et résolues par le conteneur Laravel.
- L'accès direct au conteneur depuis le domaine applicatif est interdit.
- Les vues Blade ne contiennent ni requête Eloquent ni règle métier ; tous les libellés sont traduisibles.
- Les recherches de dossiers autorisés et de pièces manquantes sont explicites et testables.
- Chaque transition utilise une transaction, un verrou sur le dossier et une vérification de version.
- Les environnements local, test, staging et production sont séparés par configuration.
- Les changements de schéma sont exclusivement réalisés par migrations.

8 Organisation attendue

L'arborescence Laravel standard est conservée. Les ajouts ci-dessous expriment les responsabilités sans transformer le framework en architecture artificielle.

Zone	Responsabilité
app/Http/Controllers	Orchestration HTTP et réponses
app/Http/Requests	Validation HTTP ; app/Politiques pour les droits
app/Models et app/Enums	Relations, casts, rôles et statuts
app/Data	DTO immuables justifiés pour création et transitions
app/Actions ou app/Services	Cas d'usage, complétude et workflow
app/Contracts	Ports utiles, notamment la stratégie Blade/JSON
app/Queries ou app/Repositories	Requêtes complexes justifiées et paginées
app/Exceptions	Échecs métier typés
app/Media	Stockage privé, diffusion et cache des pièces
app/Observers Events Listeners	Réactions après commit et audit technique idempotent
database/migrations factories seeders	Schéma et données fictives
resources/views et lang/fr lang/wo	Blade et traductions
tests/Unit Feature Integration	Tests isolés, HTTP et services réels
docker et compose	Construction et orchestration
docs/devlogs	Journal par incrément et décisions

9 Modèle de données

L'étudiant produit un diagramme de classes et un schéma relationnel avant de coder les relations. Il justifie les types SQL, index, clés étrangères, règles de suppression et contraintes d'unicité. Le schéma évolue uniquement par migrations.

Décision attendue L'usage d'un Repository n'est pas automatique. L'étudiant doit comparer Eloquent direct, Query Object et Repository puis retenir la solution la plus simple qui protège réellement les règles et les tests.

10 Périmètre et droits d'accès

Le socle couvre le catalogue administratif, le dépôt et le traitement de dossiers privés. Les comptes, rôles et administrations servent à protéger ce socle. Les trois évolutions fonctionnelles sont le workflow avancé avec historique, Français/Wolof et les thèmes clair/sombre. Le portail est un monolithe Laravel ; MinIO et Redis sont des services techniques, pas des microservices métier.

Acteurs et responsabilités

Acteur	Lecture	Mutations autorisées
Visiteur	Catalogue actif	Aucune mutation de dossier
CITOYEN	Ses dossiers, pièces et historique	Créer et modifier son brouillon ; soumettre ; compléter ; annuler selon la matrice
AGENT	Dossiers soumis de son administration	Prendre en étude un dossier libre ou déjà affecté à lui ; demander un complément ; décider ses dossiers
ADMIN	Catalogue et comptes administratifs	Configurer administrations, services, démarches et agents ; aucune décision sur dossier par ce rôle seul

L'inscription publique attribue uniquement CITOYEN. ADMIN crée ou habilite les agents et leur rattachement administratif. Le premier administrateur est créé par une commande sécurisée documentée ; aucun mot de passe réel ne figure dans un seeder ou une image. Un utilisateur possède un rôle unique pour cette version.

Politiques et refus attendus

- DossierPolicy protège view, update, submit, cancel, takeInStudy, requestComplement et decide. PieceJustificativePolicy protège upload, replace, remove et download ; Gate manageCatalog protège la configuration.
- Les listes sont filtrées avant pagination ; une Policy est aussi appliquée au détail, à chaque mutation et à chaque émission d'URL. Un identifiant deviné ne contourne pas l'autorisation.
- L'agent ne voit pas les brouillons. Le rôle ADMIN ne donne pas accès par défaut aux fichiers citoyens. Un compte désactivé ne peut plus accéder aux dossiers.

Limites fonctionnelles

La V1 ne comprend ni paiement, QR Code, identité biométrique ou fédérée, connexion aux systèmes de l'État, génération de document officiel, notifications email/SMS, formulaires dynamiques, moteur de recherche intelligent ou tableaux de bord analytiques. Les conditions et délais du catalogue sont indicatifs et fictifs. Le suivi repose sur le statut, le motif et l'historique consultables dans le portail.

11 Schéma relationnel et contrats de données

Les tables ont une clé primaire BIGINT, created_at et updated_at sauf justification. Les codes de rôles et de statuts sont des enums PHP adossées à des colonnes contraintes. Les identifiants publics restent soumis aux Politiques ; ils ne constituent pas un mécanisme de sécurité.

Table	Champs essentiels	Relations et contraintes
users	name, email, password, role, active, administration_id nullable, preferred_locale fr/wo	email unique ; AGENT exige administration_id ; mot de passe haché
administrations	code, nom_fr, nom_wo, active	code unique ; désactivation au lieu de suppression
services_publics	administration_id, code, nom_fr, nom_wo, active	FK administration ; unique administration_id/code
demarches	service_public_id, code, titre et description fr/wo, conditions fr/wo, delai_indicatif_jours, active	code unique ; délai entier positif ou nul ; suppression restrictive
exigences_pieces	demarche_id, code, libelle_fr, libelle_wo, obligatoire, active	unique demarche_id/code ; aucune suppression si une pièce la référence
dossiers	reference, citoyen_id, demarche_id, administration_id, agent_id nullable, objet, informations, statut, version, demarche_snapshot JSON nullable, soumis_at, termine_at	reference unique ; FK restrictives ; version entière positive ; administration fixée à la création
dossier_histories	dossier_id, auteur_id, de_statut nullable, vers_statut, motif nullable, version, occurred_at UTC	unique dossier_id/version ; journal append-only ; auteur issu de session
pieces_justificatives	dossier_id, exigence_id, version, active, auteur_id, nom_original, mime_type, taille, disque, cle_stockage	cle_stockage unique ; unique dossier_id/exigence_id/version ; anciennes versions conservées

Historisation et stabilité

La création produit BROUILLON, version 1 et une ligne d'historique avec de_statut nul. Chaque transition augmente la version et ajoute une ligne. Les mutations de champs ou pièces augmentent aussi la version du dossier pour invalider les formulaires périmés ; leur audit technique est distinct et les versions d'historique peuvent donc comporter des intervalles.

À la première soumission, copier dans demarche_snapshot les intitulés, conditions, délai et exigences de la démarche, avec leurs identifiants. Les compléments utilisent ensuite cette copie. Les changements du catalogue ne réécrivent pas les conditions d'un dossier déjà soumis. Une modification de rattachement d'une démarche utilisée est refusée ; désactiver puis créer une nouvelle entrée si nécessaire.

- Index composés recommandés : dossiers(citoyen_id, created_at), dossiers(administration_id, statut, created_at), dossiers(agent_id, statut), dossier_histories(dossier_id, occurred_at), pieces_justificatives(dossier_id, exigence_id, active).
- Ne jamais supprimer en cascade citoyens, dossiers, historique ou fichiers. Les mutations du catalogue et les opérations critiques prennent les verrous nécessaires pour que le snapshot et la complétude correspondent à la même version.

12 Matrice de workflow des dossiers

Tous les états sont stockés exactement avec les codes ci-dessous. Les libellés affichés sont traduits, sans changer les valeurs en base. Toute transition absente de la matrice est interdite ; aucun changement de statut direct via update générique n'est accepté.

État initial	État cible	Acteur et condition
BROUILLON	SOUMIS	Propriétaire actif ; catalogue actif ; informations et pièces obligatoires complètes
BROUILLON	ANNULE	Propriétaire ; conservation du dossier
SOUMIS	EN_ETUDE	Agent de l'administration ; dossier libre ou déjà affecté à lui ; affectation sous verrou
SOUMIS	ANNULE	Propriétaire ; aucune décision ni prise en étude préalable
EN_ETUDE	COMPLEMENT_REQUIS	Agent affecté ; motif explicite de 5 à 2000 caractères
EN_ETUDE	VALIDE	Agent affecté ; dossier complet ; aucune pièce obligatoire manquante
EN_ETUDE	REJETE	Agent affecté ; motif explicite de 5 à 2000 caractères
COMPLEMENT_REQUIS	SOUMIS	Propriétaire ; compléments déposés ; exigences du snapshot revérifiées
COMPLEMENT_REQUIS	ANNULE	Propriétaire ; pièces et historique conservés

Comportement par état

- BROUILLON : objet, informations et pièces modifiables par le citoyen. La démarche et l'administration du dossier ne sont pas modifiées ; démarrer un autre brouillon pour une autre démarche.
- SOUMIS : lecture et annulation citoyenne, prise en étude agent ; les pièces et champs sont gelés.
- EN_ETUDE : lecture citoyenne, complément ou décision par l'agent affecté ; aucune annulation citoyenne pendant l'étude.
- COMPLEMENT_REQUIS : le motif décrit ce qui manque ; le propriétaire corrige champs et pièces puis resoumet. L'agent affecté reste le même. Il reprend ensuite le dossier SOUMIS en EN_ETUDE.
- VALIDE, REJETE et ANNULE : états terminaux en lecture seule. Aucune réouverture, nouvelle décision ou suppression ; VALIDE désigne une décision pédagogique, sans document officiel délivré.

Motifs et historique

Le motif de complément ou rejet est public pour le propriétaire et l'agent autorisé. Il ne contient pas de note interne sensible. L'annulation et la validation peuvent recevoir un motif facultatif. L'historique présente date, libellé de transition, acteur autorisé et motif ; il n'expose ni email privé d'agent ni information d'autres dossiers.

Une soumission répétée depuis un formulaire déjà traité est refusée comme version périmée. Elle ne crée pas de seconde ligne d'historique. Les tests couvrent les 49 paires d'états et les trois rôles, avec propriétaire et rattachement étrangers.

13 Frontière applicative et atomicité

Cycle d'une transition

Route nommée → session et middleware auth → Form Request → Policy → DTO immuable → Action de workflow → transaction MySQL et verrou → statut et historique → événement après commit → réponse Blade traduite. La représentation JSON du catalogue de pièces utilise les mêmes contrôles.

- TransitionDossierData contient targetStatus, expectedVersion et motif nullable. L'acteur authentifié est un argument séparé. Les enums et types sont validés avant l'action, puis les invariants sont contrôlés dans l'action.
- Démarrer la transaction et verrouiller le dossier par sa clé primaire. Relire statut, version, affectation et permissions. Si expectedVersion diffère, lever VersionDossierObsolete sans écrire.
- Prendre les autres verrous dans un ordre constant : dossier, démarche si première soumission, pièces. Toutes les mutations de champs et pièces utilisent le verrou de dossier et augmentent sa version.
- Vérifier la transition et les pièces actives ; fixer agent_id si prise en étude ; écrire statut, version, dates et DossierHistory ensemble. Le premier soumis_at n'est pas remplacé lors d'une resoumission ; termine_at est fixé seulement en état terminal.
- Après commit, émettre DossierStatusChanged pour un audit technique si nécessaire. Le journal métier est déjà écrit dans la transaction, jamais créé uniquement par un Listener.

Erreurs et réponses

Échec	Réponse HTML	Résultat sur les données
Entrées invalides	Redirection avec erreurs et anciennes valeurs	Aucune mutation
Accès refusé	403	Aucune donnée ou URL divulguée
Ressource absente	404	Aucune mutation
Transition interdite ou version périmée	Message de conflit et invitation à recharger	Statut et historique inchangés ; JSON 409
Dossier incomplet ou démarche inactive	Retour métier ciblé au formulaire	Brouillon ou complément conservé
Échec technique	Message générique ; identifiant de diagnostic	Rollback ; aucun secret affiché

Test réel de concurrence

Créer un dossier EN_ETUDE affecté à un agent et ouvrir deux connexions MySQL indépendantes. Les deux actions portent la même expectedVersion, l'une vise VALIDE et l'autre REJETE. Synchroniser leur départ avec une barrière. Vérifier une réussite, un conflit, une hausse de version et une seule nouvelle ligne d'historique. Reproduire la course prise en étude contre annulation et mutation de pièce contre soumission.

La reprise d'un deadlock est bornée et rejoue les contrôles sous verrou. Ne pas exécuter d'écriture MinIO ou d'effet non idempotent dans une closure de transaction automatiquement rejouée.

14 Français Wolof et préférences visuelles

Langues et traductions

Le français est la langue initiale et de repli. Le Wolof est sélectionnable sur toutes les pages du portail. Utiliser les codes fr et wo ; ne jamais employer le choix utilisateur comme chemin de fichier. Les statuts, rôles, routes et clés de base restent indépendants de la langue.

- Publier le répertoire lang si nécessaire avec php artisan lang:publish. Prévoir lang/fr et lang/wo avec fichiers navigation.php, dossiers.php, démarches.php, validation.php, auth.php et messages.php.
- Utiliser __('dossiers.status.EN_ETUDE') et des paramètres nommés pour les messages. Traduire navigation, boutons, titres, aide, erreurs, notifications flash et libellés des sept statuts. Le catalogue possède les champs fr/wo définis dans le modèle.
- Configurer locale=fr et fallback_locale=fr dans config/app.php. Un middleware applique la locale à chaque requête avant validation et rendu : preferred_locale du compte, puis préférence de session pour un visiteur, puis fr.
- POST /preferences/langue valide uniquement fr ou wo ; avec session et CSRF, enregistrer en compte connecté ou en session visiteur. À la connexion, la préférence du compte prévaut ; si elle n'existe pas encore, initialiser depuis la session valide.
- Les seeders proposent un catalogue fictif dans les deux langues. Un locuteur compétent relit les traductions Wolof avant la recette v1.0.0 ; une clé absente doit se replier en français sans afficher la clé brute.

Thèmes clair sombre et système

Prévoir trois choix de préférence : système, clair et sombre. Deux thèmes sont rendus. Sans choix mémorisé, suivre prefers-color-scheme. Un choix manuel clair ou sombre prévaut sur le système ; le choix système rétablit le suivi automatique.

- Mémoriser la préférence avec une clé e-senegal-theme dans localStorage ; valider ses valeurs. Si le stockage échoue, utiliser une préférence en mémoire et le système sans bloquer le portail.
- Appliquer data-theme sur l'élément html avant l'affichage pour éviter un flash ; gérer le script dans la politique CSP retenue. Observer les changements du système uniquement quand le mode système est actif.
- Définir des variables CSS pour fond, texte, bordures, champs, erreurs et focus. Les statuts portent un libellé, jamais une couleur seule ; le contrôle de thème a un nom accessible et fonctionne au clavier.

Preuves de recette

Vérifier catalogue, formulaire invalide, pièces, historique et préférences en fr et wo, en clair et sombre, sur 360 px et sur ordinateur. Recharger, ouvrir une nouvelle session et changer la préférence système. Les tests automatisés vérifient résolution de locale et persistance ; les captures complètent les preuves pour alignement, focus et lisibilité. Aucun texte du portail ne prétend être une traduction administrative officielle.

15 Pièces privées et contrats HTTP

Validation et cycle de stockage

UploadPieceRequest vérifie le type MIME réel PDF/JPEG/PNG, 5 Mio maximum, la taille PHP et serveur cohérente, le dossier, l'exigence attendue et le nombre de pièces courantes. Le nom original sert seulement à l'affichage échappé ; le serveur génère une clé aléatoire indépendante du nom.

Déposer l'objet privé puis, sous verrou du dossier, vérifier version, permission et état avant de créer la métadonnée. En cas d'échec MySQL, supprimer l'objet nouvellement déposé. Si la suppression échoue, journaliser une reprise technique sans secret et fournir une commande de réconciliation des objets orphelins avec délai de sécurité. Ne pas supprimer un objet antérieur avant qu'un remplacement soit validé.

Une pièce remplacée ou retirée devient inactive ; la nouvelle version reste la seule active pour son exigence. Le verrou de dossier protège cet invariant. Le retrait logique conserve l'ancienne preuve ; une suppression physique de fichier de test ou d'objet orphelin est un cas technique distinct. Le projet ne définit pas une politique réelle de destruction de données citoyennes.

- Après commit, PieceJustificativeObserver publie DossierPiecesChanged une fois par mutation logique. Les Listeners invalident pieces:dossier:{id} et écrivent un audit idempotent. Les bulk updates Eloquent contournant les Observers sont interdits pour ces mutations.
- Le cache stocke uniquement des métadonnées non signées, TTL indicatif cinq minutes. L'accès est contrôlé même sur cache hit. Chaque changement invalide immédiatement ; le TTL ne remplace pas l'invalidation.
- La route de téléchargement autorise l'acteur puis émet une URL signée de cinq minutes ou transmet un flux privé. Une URL signée est un jeton porteur utilisable jusqu'à expiration ; utiliser le flux Laravel si l'autorisation doit être revérifiée à chaque accès.

Routes et conventions

Route indicative	Verbe	Protection
/demarches et /demarches/{demarche}	GET	Catalogue actif public
/dossiers et /dossiers/{dossier}	GET	auth, requête filtrée et DossierPolicy
/dossiers	POST	auth, rôle citoyen, StoreDossierRequest
/dossiers/{dossier}	PATCH	auth, propriétaire, état modifiable et version
/dossiers/{dossier}/transitions	POST	auth, TransitionDossierRequest, Policy et Action
/dossiers/{dossier}/pieces	POST	auth, UploadPieceRequest, Policy et Action
/pieces/{piece}/telecharger	GET	auth, PieceJustificativePolicy ; jamais URL publique
/pieces/{piece}	DELETE	auth, retrait logique autorisé et version
/preferences/langue	POST	session, CSRF et locale fr/wo
/admin/administrations services demarches agents	GET POST PATCH	auth et Gate manageCatalog

Les formulaires de transition portent expectedVersion. Les routes d'administration sont séparées et nommées ; aucune mutation ne se fait en GET. APP_DEBUG=false en staging et production ; masquer chemins, clés et données citoyennes dans les journaux.

B Démarche incrémentale

Incrément 0 Initialisation du dépôt GitLab

Version cible	Résultat observable	Condition de passage
v0.0.0	Un dépôt vide mais gouverné, clonable et auditable.	La Merge Request est fusionnée, le pipeline est vert et le tag v0.0.0 pointe sur main.

Travail demandé

- Créer le projet GitLab et un README présentant contexte, périmètre et prérequis.
- Créer les labels type, priorité et statut ainsi que les Issues des trois premiers incréments.
- Protéger main et interdire le push direct.
- Ajouter des modèles de Merge Request, Issue et DevLog.

Contraintes techniques

- main représente toujours un état livrable.
- Toute évolution passe par une branche issue incrément sujet.
- Les secrets et fichiers locaux sont exclus dès le premier commit.

Questions de découverte

10. Pourquoi protéger main avant d'écrire du code.
11. Quelle différence entre commit, tag, release et environnement.
12. Quels éléments rendent un commit atomique.

GitLab imposé

- Branche chore 0 initialisation.
- Commits suggérés chore gitlab initialise project governance puis docs readme describe project.
- Merge Request obligatoire, squash autorisé si l'historique de travail reste lisible.
- Tag annoté v0.0.0 après fusion.

Docker évolutif

- Créer seulement docs docker strategy.md avec les futures cibles local, ci et production.
- Aucun conteneur n'est exigé à ce stade.

Pipeline et déploiement

- Créer un pipeline minimal lint markdown ou vérification de structure.
- Le pipeline doit s'exécuter sur la Merge Request et main.

DevLog obligatoire

- Créer docs devlogs v0.0.0.md.
- Expliquer la convention de branches, la protection de main et la première difficulté rencontrée.
- Ajouter les liens vers l'Issue, la Merge Request, le pipeline et le tag.

Validation de l'incrément La Merge Request est fusionnée, le pipeline est vert et le tag v0.0.0 pointe sur main.

Incrément 1 Socle Laravel exécutable

Version cible	Résultat observable	Condition de passage
v0.1.0	La page d'accueil Laravel répond localement et les tests du framework passent.	Un clone propre peut installer, démarrer et tester le socle avec les commandes documentées.

Travail demandé

- Créer le projet Laravel avec Composer.
- Configurer APP_NAME=e-Sénégal et créer une page d'accueil décrivant le catalogue, le dépôt et le suivi.
- Identifier les répertoires clés et décrire le cycle route, middleware, Form Request, contrôleur, action, réponse Blade.
- Ajouter Pint et l'outil d'analyse statique choisi.

Contraintes techniques

- composer.lock est versionné.
- .env n'est jamais versionné et .env.example ne contient aucun secret.
- La version PHP et les extensions requises sont documentées.

Questions de découverte

13. Quel fichier reçoit réellement la requête HTTP.
14. À quoi servent Artisan, Composer et les Service Providers.
15. Pourquoi figer les dépendances directes et transitives.

GitLab imposé

- Branche feat 1 bootstrap laravel.
- Commits suggérés build composer create laravel application puis feat home expose project entry page.
- Tag annoté v0.1.0 après revue.

Docker évolutif

- Ajouter un Dockerfile dev simple exécutant PHP et Composer.
- Le bind mount autorise le rechargement du code local.
- Documenter la différence entre image, conteneur, volume et port.

Pipeline et déploiement

- Étapes composer validate, composer install, Pint check, analyse statique et tests.
- Mettre en cache les dépendances sans mettre vendor dans le dépôt.

DevLog obligatoire

- Décrire l'amorçage Laravel, la place de public/index.php et le rôle du lockfile.
- Comparer une exécution locale native et conteneurisée.

Validation de l'incrément Un clone propre peut installer, démarrer et tester le socle avec les commandes documentées.

Incrément 2 Environnement applicatif et MySQL

Version cible	Résultat observable	Condition de passage
v0.2.0	Laravel communique avec MySQL dans un environnement local reproductible.	Le pipeline établit une connexion MySQL et les commandes de diagnostic réussissent dans un clone propre.

Travail demandé

- Ajouter MySQL à Docker Compose et configurer la connexion par variables d'environnement.
- Ajouter un healthcheck MySQL et une commande de diagnostic.
- Créer un environnement de test distinct.

Contraintes techniques

- Aucune adresse de service n'est codée en dur dans PHP.
- Les données MySQL persistent dans un volume nommé.
- La base de test est isolée de la base de développement.

Questions de découverte

16. Pourquoi depends on ne garantit pas que MySQL accepte déjà des connexions.
17. Quelle différence entre configuration construite et configuration injectée.
18. Pourquoi config cache peut provoquer un comportement surprenant.

GitLab imposé

- Branche feat 2 mysql environment.
- Commits suggérés build docker compose add mysql puis config database isolate environments.
- Tag v0.2.0.

Docker évolutif

- Services app et db, réseau privé, volume nommé et healthcheck.
- Ajouter une cible Makefile ou scripts Composer pour up, down, logs et shell.

Pipeline et déploiement

- Ajouter un service MySQL au job d'intégration ou utiliser une image de service.
- Le job attend explicitement la disponibilité de la base.

DevLog obligatoire

- Présenter le flux de configuration du fichier env jusqu'à PDO.
- Documenter un incident de démarrage ou de connexion et son diagnostic.

Validation de l'incrément Le pipeline établit une connexion MySQL et les commandes de diagnostic réussissent dans un clone propre.

Incrément 3 Schéma et modèles Eloquent

Version cible	Résultat observable	Condition de passage
v0.3.0	Le schéma, les modèles et leurs relations représentent correctement le domaine.	Les migrations montent sur une base vide, redescendent si prévu, puis remontent dans le pipeline.

Travail demandé

- Créer les migrations users, administrations, services_publics, demarches, exigences_pieces, dossiers, dossier_histories et pieces_justificatives.
- Créer les modèles User, Administration, ServicePublic, Demarche, ExigencePiece, Dossier, DossierHistory et PieceJustificative avec leurs relations.
- Définir les casts Role et DossierStatus, les champs modifiables et les dates ; interdire l'assignation de masse de role, citoyen_id et statut.
- Produire le diagramme de classes, le schéma relationnel et la matrice des transitions ; intégrer les préférences de langue.

Contraintes techniques

- Clés étrangères, index et contraintes sont justifiés.
- Une migration existante fusionnée n'est pas réécrite pour corriger un environnement partagé.
- Les états métier sont modélisés par enum PHP si le choix est compatible avec la stratégie SQL.

Questions de découverte

19. Pourquoi une relation Eloquent n'est-elle pas la même chose qu'une clé étrangère.
20. Quels index servent les listes citoyen et agent, la recherche des pièces requises et l'historique.
21. Quel compromis entre enum SQL, chaîne contrainte et enum PHP.

GitLab imposé

- Branche feat/3-administrative-schema.
- Commits séparés pour migrations, modèles relations et documentation du modèle.
- Tag v0.3.0.

Docker évolutif

- Ajouter une commande reproductible migrate fresh.
- Ne pas embarquer de données MySQL dans l'image.

Pipeline et déploiement

- Exécuter migrate fresh dans le job d'intégration.
- Publier le journal de migration en artefact seulement en cas d'échec.

DevLog obligatoire

- Justifier types, index, relation un à plusieurs, casts et stratégie de suppression.
- Inclure les diagrammes versionnés.

Validation de l'incrément Les migrations montent sur une base vide, redescendent si prévu, puis remontent dans le pipeline.

Incrément 4 Jeu de données reproductible

Version cible	Résultat observable	Condition de passage
v0.4.0	Une base réaliste est générée sans saisie manuelle.	Une seule commande reconstruit une base cohérente et les relations se vérifient par tests.

Travail demandé

- Créer factories et seeders pour administrations, services et démarches actifs et inactifs.
- Créer des dossiers dans les sept états, avec historique cohérent, pièces présentes et cas de complément manquant.
- Créer des comptes CITOYEN, AGENT et ADMIN fictifs, ainsi que deux administrations permettant de tester le cloisonnement.

Contraintes techniques

- Les données de test sont déterministes lorsque le scénario l'exige.
- Aucun secret ni donnée personnelle réelle.
- Factories et seeders gardent des responsabilités distinctes.

Questions de découverte

22. Pourquoi les factories sont utiles aux tests au-delà du seeding.
23. Quand rendre un jeu aléatoire déterministe.
24. Comment éviter des scénarios impossibles produits par Faker.

GitLab imposé

- Branche feat 4 seed demo data.
- Commits test factories model valid states et feat seed development catalog.
- Tag v0.4.0.

Docker évolutif

- La commande de bootstrap local enchaîne migration et seed explicitement.
- Le seed de développement n'est jamais lancé automatiquement en production.

Pipeline et déploiement

- Le pipeline fabrique ses propres données avec factories.
- Aucun artefact de base de données n'est réutilisé entre pipelines.

DevLog obligatoire

- Expliquer factory, state, seeder et idempotence.
- Présenter deux cas limites rendus testables grâce aux données générées.

Validation de l'incrément Une seule commande reconstruit une base cohérente et les relations se vérifient par tests.

Incrément 5 Validation des entrées HTTP

Version cible	Résultat observable	Condition de passage
v0.5.0	Les formulaires invalides sont refusés avec des messages utiles.	Les cas invalides prévus retournent les erreurs attendues et aucun enregistrement n'est créé.

Travail demandé

- Créer StoreDemarcheRequest, StoreDossierRequest, UpdateDossierRequest, TransitionDossierRequest et UploadPecRequest.
- Valider champs, formats, longueurs, enums, version attendue et MIME réel ; séparer règles de création, brouillon et soumission.
- Préparer les messages dans lang/fr et lang/wo ; le français reste la langue de repli.
- Conserver les anciennes valeurs et afficher les erreurs dans Blade.

Contraintes techniques

- La validation de forme ne décide pas seule de la transition ou de la complétude du dossier.
- Le contrôleur n'accède qu'aux données validées ; rôle, propriétaire, administration et statut sont déterminés côté serveur.
- Mettre en place l'authentification par session, les middlewares auth et les Politiques/Gates ; une requête forgée doit être refusée.

Questions de découverte

25. Pourquoi toutes les règles ne doivent pas être placées dans un Form Request.
26. Quelle différence entre validation syntaxique et invariant métier.
27. Quand utiliser Rule enum, after validation ou une règle personnalisée.

GitLab imposé

- Branche feat/5-validation-and-access.
- Commits séparés pour demandes, messages et tests de validation.
- Tag v0.5.0.

Docker évolutif

- Aucune modification structurelle obligatoire.
- Vérifier que les erreurs sont visibles derrière le serveur conteneurisé.

Pipeline et déploiement

- Ajouter les Feature tests de validation à la Quality Gate.
- Le pipeline échoue si une route accepte une valeur interdite.

DevLog obligatoire

- Classer les règles entre forme, autorisation et métier.
- Documenter un choix de règle Laravel et une alternative rejetée.

Validation de l'incrément Les cas invalides prévus retournent les erreurs attendues et aucun enregistrement n'est créé.

Incrément 6 Objets de données et frontière applicative

Version cible	Résultat observable	Condition de passage
v0.6.0	Le cas d'usage reçoit des données typées indépendantes de la requête HTTP.	Le service peut être testé sans construire de requête HTTP et rejette les objets incohérents.

Travail demandé

- Créer CreateDossierData et TransitionDossierData immuables ; justifier leur utilité pour les cas d'usage appelés depuis HTTP et tests.
- Transformer les données validées en valeurs typées ; la cible est un DossierStatus et expectedVersion est un entier.
- Définir TransitionInterdite, DossierIncomplet, DemarcheIndisponible et VersionDossierObsolete.

Contraintes techniques

- Aucun Request n'entre dans un service métier.
- Les objets de données sont immuables.
- L'identité de l'acteur est injectée depuis l'authentification, jamais recopiée d'un champ client ; les dates sont converties une seule fois.

Questions de découverte

28. Quel couplage apparaît lorsqu'un service reçoit Illuminate Http Request.
29. Quand un DTO est utile et quand il ajoute seulement du bruit.
30. Pourquoi une exception métier ne doit pas dépendre du texte affiché à l'utilisateur.

GitLab imposé

- Branche refactor 6 application data boundary.
- Commits feat(data): add typed dossier inputs puis feat(domain): add business exceptions.
- Tag v0.6.0.

Docker évolutif

- Aucune évolution requise.
- Vérifier que le fuseau du conteneur est cohérent avec la convention applicative.

Pipeline et déploiement

- Tests unitaires de construction et de conversion des objets de données.
- Conserver les mêmes Quality Gates.

DevLog obligatoire

- Illustrer le passage HTTP vers données validées puis objet applicatif.
- Expliquer immutabilité, typage et gestion du fuseau.

Validation de l'incrément Le service peut être testé sans construire de requête HTTP et rejette les objets incohérents.

Incrément 7 Accès aux données et injection

Version cible	Résultat observable	Condition de passage
v0.7.0	Les requêtes de visibilité et de complétude sont explicites et injectées.	Les tests distinguent propriétaire, autre citoyen, administration étrangère et pièces manquantes.

Travail demandé

- Créer `DossiersVisiblesQuery` et `PiecesManquantesQuery` pour le cloisonnement, la pagination et la complétude.
- Choisir Eloquent direct, Query Object ou Repository ; justifier le choix sur ces deux requêtes au lieu d'un Repository par modèle.
- Créer un Contract seulement pour une dépendance interchangeable réelle ; la stratégie Blade/JSON de l'incrément 12 fournit un cas concret.
- Configurer les bindings nécessaires dans un Service Provider.

Contraintes techniques

- Pas de façade globale dans le service métier.
- Pas de Service Locator ni d'appel `app` dans les classes applicatives.
- Les requêtes de dossiers autorisés et de pièces manquantes restent lisibles et couvertes par MySQL ; les Policies restent indispensables.

Questions de découverte

31. Comment traduire le cloisonnement par propriétaire ou administration dans une requête SQL.
32. Pourquoi masquer Eloquent derrière un Repository peut parfois être une abstraction trompeuse.
33. Que réalise réellement le conteneur lors de l'autowiring.

GitLab imposé

- Branche `feat/7-dossier-queries`.
- Commits `feat(query): scope visible dossiers` puis `test(integration): cover administration boundaries`.
- Tag `v0.7.0`.

Docker évolutif

- Aucune évolution requise.
- Utiliser MySQL pour le test d'intégration qui dépend du comportement SQL réel.

Pipeline et déploiement

- Ajouter un job integration avec MySQL.
- Les tests unitaires peuvent rester sans base.

DevLog obligatoire

- Comparer les trois stratégies d'accès aux données.
- Expliquer Dependency Inversion, binding et autowiring sur le cas réel.

Validation de l'incrément Les requêtes ne retournent aucun dossier d'un autre citoyen ou d'une autre administration ; les pièces requises manquantes sont identifiées.

Incrément 8 Actions et workflow des dossiers

Version cible	Résultat observable	Condition de passage
v0.8.0	Les actions appliquent les règles et toutes les transitions du dossier.	Les neuf règles métier, les transitions autorisées et les états terminaux sont couverts par des tests verts.

Travail demandé

- Créer CreateDossier, SoumettreDossier et TransitionDossier ; centraliser la matrice dans un service de workflow testé.
- Vérifier les permissions, la démarche, l'état courant, la version, la complétude et le motif selon la transition.
- Changer le statut et ajouter DossierHistory dans une transaction ; publier DossierStatusChanged seulement après commit.
- Implémenter l'annulation autorisée et définir le comportement d'un doublon : version périmée refusée, aucune seconde ligne d'historique.

Contraintes techniques

- Le contrôleur n'exécute aucune décision métier et ne modifie jamais directement le statut.
- L'ordre des contrôles est justifié ; l'autorisation est vérifiée avant d'exposer un détail de dossier.
- Un état terminal n'admet aucune sortie ; la demande de complément conserve la même affectation d'agent.

Questions de découverte

34. Quel principe SOLID est visible dans la séparation du cas d'usage.
35. Pourquoi une suite de règles correcte en mémoire peut échouer sous concurrence.
36. Quel résultat retourner au contrôleur sans lui exposer les détails internes.

GitLab imposé

- Branche feat/8-dossier-workflow.
- Un commit par groupe cohérent création, soumission, compléments, décisions, historique et tests.
- Tag v0.8.0.

Docker évolutif

- Aucune évolution requise.
- Les tests métier doivent pouvoir tourner dans le conteneur app.

Pipeline et déploiement

- Séparer les suites unit et integration si leur coût diffère.
- Publier un rapport JUnit exploitable par GitLab.

DevLog obligatoire

- Associer chaque transition et chaque interdiction à un test et à un résultat.
- Expliquer au moins une responsabilité déplacée hors du contrôleur.

Validation de l'incrément Les neuf règles métier, les transitions permises et interdites, les motifs et l'historique sont testés.

Incrément 9 Interface Blade bilingue et thèmes

Version cible	Résultat observable	Condition de passage
v0.9.0	Le portail Blade est utilisable en français et Wolof et en thèmes clair et sombre.	Les parcours citoyen, agent et admin, les erreurs 403/404/405 et les préférences sont démontrés.

Travail demandé

- Déclarer des routes nommées et appliquer le route model binding sous protection des Politiques.
- Créer contrôleurs minces, layout Blade et espaces citoyen, agent et administration ; pas de SPA React ou Vue.
- Implémenter catalogue, dossiers, pièces et historique, puis sélection Français/Wolof et thèmes clair/sombre/système.
- Gérer 404, 403 et conflit métier ; montrer le motif public d'un complément ou rejet et une confirmation de mutation.

Contraintes techniques

- Aucune requête Eloquent dans Blade.
- Protection CSRF sur toute mutation.
- Les verbes HTTP correspondent à l'intention.
- Tous les libellés et messages utilisent des clés lang ; aucune URL signée ni information privée dans un message flash.

Questions de découverte

37. Quelle différence entre erreur de validation, conflit métier et ressource absente.
38. Pourquoi utiliser des routes nommées et le route model binding.
39. Comment appliquer une préférence linguistique sans modifier les enums stockées ni le format des dates en base.

GitLab imposé

- Branche feat/9-blade-languages-and-themes.
- Commits par verticale fonctionnelle plutôt que par type de fichier.
- Tag v0.9.0.

Docker évolutif

- Ajouter un service web dédié si l'image app utilise PHP FPM.
- Documenter ports, réseau interne et exposition publique.

Pipeline et déploiement

- Ajouter des Feature tests HTTP et de locale ainsi qu'une vérification visuelle des deux langues et deux thèmes.
- Conserver captures ou traces de scénarios en artefacts de courte durée si utile.

DevLog obligatoire

- Tracer une requête jusqu'à Blade, puis la résolution de la langue et du thème ; joindre captures ordinateur et mobile.
- Expliquer la gestion des statuts HTTP et du CSRF.

Validation de l'incrément Tous les parcours sont utilisables dans les deux langues et thèmes ; les refus d'accès et réponses HTTP sont corrects.

Incrément 10 Stratégie de tests et qualité

Version cible	Résultat observable	Condition de passage
v0.10.0	La non régression est automatisée et mesurable.	Les Quality Gates sont obligatoires et les scénarios critiques sont automatisés.

Travail demandé

- Compléter les tests Unit et Feature selon une pyramide justifiée.
- Couvrir la matrice complète des états, les refus d'accès, les pièces obligatoires et les erreurs métier.
- Configurer couverture, Pint et analyse statique ; les intégrations MySQL contrôlent les transactions réelles.
- Fixer un seuil progressif documenté ; toutes les transitions permises et interdites et les frontières d'accès ont des tests, indépendamment du pourcentage.

Contraintes techniques

- Les tests sont indépendants, reproductibles et lisibles.
- Pas de test qui dépend de l'ordre d'exécution.
- Les données sont créées par factories ou builders.

Questions de découverte

40. Qu'est ce qui rend un test unitaire plutôt qu'un test d'intégration.
41. Pourquoi cent pour cent de couverture ne garantit pas la qualité.
42. Quels tests apportent la meilleure protection contre une régression métier.

GitLab imposé

- Branche test 10 complete quality strategy.
- Commits par famille de scénarios et configuration qualité.
- Tag v0.10.0.

Docker évolutif

- Créer une cible test identique pour poste local et pipeline.
- Éviter qu'un bind mount masque les dépendances installées dans l'image CI.

Pipeline et déploiement

- Stages validate, quality, test et integration.
- Rapports JUnit et couverture exposés dans GitLab.
- La Merge Request est bloquée si un job requis échoue.

DevLog obligatoire

- Présenter la pyramide retenue et les doublures utilisées.
- Analyser une régression que la suite aurait détectée.

Validation de l'incrément Les Quality Gates sont obligatoires et les scénarios critiques sont automatisés.

Incrément 11 Transactions et décisions concurrentes

Version cible	Résultat observable	Condition de passage
v0.11.0	Deux décisions contradictoires sur un même dossier ne peuvent pas réussir.	Sous deux connexions MySQL, une seule transition, un seul historique et une seule hausse de version sont constatés.

Travail demandé

- Identifier la course entre lecture du statut, décision et écriture de l'historique.
- Verrouiller la ligne dossiers avec lockForUpdate dans une transaction, puis relire et comparer expectedVersion avant les règles.
- Incrémenter la version avec le statut et l'historique ; toutes les mutations de pièces prennent le même verrou de dossier.
- Tester deux connexions MySQL simultanées tentant VALIDE et REJETE depuis une même version EN_ETUDE.

Contraintes techniques

- Une simple lecture du statut avant update ou une transaction sans verrou ne suffit pas.
- Documenter l'ordre des verrous dossier puis pièces et interdire les mises à jour de statut hors action.
- Les deadlocks donnent lieu à une reprise bornée de transaction ; la version est revérifiée et les effets externes attendent le commit.

Questions de découverte

43. Pourquoi une transaction seule ne supprime pas nécessairement la course.
44. Quelle donnée peut servir de point de verrouillage stable.
45. Quel compromis entre pessimistic locking, optimistic locking et contrainte de base.

GitLab imposé

- Branche fix/11-prevent-conflicting-decisions.
- Commits test: reproduce concurrent decisions puis fix(workflow): lock and version dossier transitions.
- Tag v0.11.0.

Docker évolutif

- Configurer MySQL avec le moteur et le niveau d'isolation documentés.
- Le scénario concurrent doit tourner avec la même famille de base qu'en production.

Pipeline et déploiement

- Ajouter un job de test de concurrence, éventuellement non parallèle avec lui même.
- Conserver les journaux en artefact en cas d'échec.

DevLog obligatoire

- Décrire la course avec une chronologie et prouver une seule décision, un seul historique et une version incrémentée.
- Expliquer transaction, isolation, verrou, deadlock et stratégie de reprise.

Validation de l'incrément Deux demandes simultanées depuis la même version produisent au plus une décision ; l'autre reçoit un conflit sans effet de bord.

Incrément 12 Justificatifs privés et stratégies de représentation

Version cible	Résultat observable	Condition de passage
v0.12.0	Les justificatifs sont privés dans MinIO, avec cache Redis et représentations Blade/JSON.	Le même catalogue autorisé est rendu dans les deux formats ; stockage, compensation, scopes et invalidation après commit sont testés.

Travail demandé

- Faire évoluer le stockage provisoire privé vers MinIO ; conserver le modèle de pièces et migrer les objets par une commande vérifiable.
- Compléter `pieces_justificatives` avec disque, clé unique, MIME réel, taille, version et auteur ; garder une seule pièce active par exigence.
- Comparer disque local privé, BLOB MySQL et stockage objet compatible S3 ; retenir MinIO pour cet exercice.
- Configurer un disque Laravel privé et un bucket MinIO sans accès anonyme ; documenter la migration du stockage local.
- Ajouter, remplacer et retirer des PDF, JPEG ou PNG en brouillon ou complément ; refuser extension trompeuse, fichiers trop grands et exigences étranges.
- Créer des scopes locaux pour administrations et démarches actives, dossiers visibles/par statut, pièces actives et ordre d'affichage.
- Créer `PieceJustificativeObserver` après commit ; il publie `DossierPiecesChanged` pour les créations et changements de pièce active.
- Créer `InvalidateDossierPiecesCache` et `WritePieceAuditLog` comme Listeners idempotents ; ne pas dupliquer l'historique des statuts.
- Créer une stratégie Blade pour l'interface et une représentation JSON du même catalogue de pièces, toujours authentifiée et autorisée.
- Sélectionner par `MEDIA_VIEW_DRIVER=blade` ou `json` dans `.env.example` puis `config/media.php` ; conserver blade pour le portail final.
- Mettre en cache les métadonnées du catalogue de pièces dans Redis avec invalidation après mutation ; ne pas mettre en cache le fichier ni son URL.

Contraintes techniques

- Le contrôleur n'appelle jamais `env` directement il lit uniquement `config media.view_driver`.
- Les deux stratégies implémentent un contrat commun utile à la substitution ; elles n'offrent aucune route publique de données personnelles.
- Une valeur inconnue est refusée explicitement ; la configuration choisit le format, jamais les droits.
- Les binaires restent hors de MySQL ; la base conserve disque et clé ; les anciennes versions restent privées et traçables.
- MinIO n'est pas un répertoire public anonyme les fichiers sont servis par URL temporaire ou flux autorisé.
- Le cache contient le catalogue et les métadonnées, jamais une URL signée au-delà de sa durée de validité.
- Observer, Event et Listener n'hébergent pas les règles métier principales du téléversement.
- L'Observer publie `DossierPiecesChanged` après commit ; les Listeners invalident le catalogue et écrivent un audit technique minimal.
- Une mutation logique publie une fois ; les Listeners sont idempotents et supportent une éventuelle relivraison.
- Les scopes restent composables, sans effet de bord et sans masquer une règle critique.

Questions de découverte

46. Quels avantages et limites présentent disque local, BLOB MySQL et MinIO pour la sauvegarde, la montée en charge et le déploiement.
47. Quelle différence existe entre un scope local, un scope global et une méthode de relation Eloquent.
48. Quand choisir un Observer plutôt qu'un Event et un Listener explicites.
49. Pourquoi un événement envoyé avant la validation de la transaction peut-il annoncer une opération finalement annulée.
50. Quel Design Pattern permet de remplacer la représentation Blade par JSON sans multiplier les conditions dans le contrôleur.
51. Pourquoi la valeur du .env doit-elle être copiée dans config media.php avant d'être utilisée par l'application.
52. Quelles limites présente une sélection globale Blade ou JSON par environnement par rapport à la négociation HTTP avec l'en-tête Accept.
53. Pourquoi un cache sans invalidation peut-il masquer un complément de dossier récemment déposé.
54. Comment coordonner MySQL et MinIO alors que les deux systèmes ne partagent pas la même transaction.
55. Comment servir un PDF privé sans charger tout le fichier en mémoire et sans révéler une clé de stockage.

GitLab imposé

- Branche feat/12-private-pieces-and-representations.
- Commits feat(pieces): add versioned metadata and scopes puis build(minio): add private storage puis feat(pieces): validate uploads puis feat(media): add blade json strategies puis feat(events): publish pieces changes puis perf(cache): invalidate catalog.
- La Merge Request contient les deux matrices de décision, les tests, le DevLog et les preuves Blade, JSON, MinIO et Redis.
- Tag annoté v0.12.0 après fusion sur main.

Docker évolutif

- Ajouter MinIO et Redis à Docker Compose avec volumes nommés, healthchecks et réseau privé.
- Ajouter une commande reproductible créant le bucket et sa politique sans intervention manuelle.
- Configurer endpoint, région, bucket, identifiants de développement et option path style de MinIO.
- Distinguer l'endpoint interne utilisé entre conteneurs de l'URL publique résoluble par le navigateur ou utiliser une route Laravel de diffusion.
- Tester successivement MEDIA_VIEW_DRIVER égal à blade puis json sans reconstruire l'image.
- Conserver les médias hors de l'image applicative et du dépôt Git.

Pipeline et déploiement

- Démarrer MySQL, MinIO et Redis dans le job d'intégration avec vérification de santé.
- Créer un bucket de test isolé par pipeline et le nettoyer même après échec.
- Exécuter une matrice de jobs avec MEDIA_VIEW_DRIVER égal à blade puis json.
- Tester scopes, téléversement, remplacement, retrait, publication unique de DossierPiecesChanged, Listeners, cache hit et invalidation.
- Vérifier qu'aucune clé MinIO n'apparaît dans les logs ou les artefacts.

DevLog obligatoire

- Comparer les trois stockages selon coût, portabilité, sécurité, sauvegarde, performance et montée en charge.
- Comparer Blade et JSON selon consommateur, format, statut HTTP, testabilité et couplage.
- Tracer la résolution depuis MEDIA_VIEW_DRIVER jusqu'au contrat de stratégie sélectionné.
- Expliquer scopes locaux et globaux, Observer, Event, Listener, dispatch après commit, Redis, TTL, invalidation et cloisonnement des clés.
- Décrire un échec partiel MySQL MinIO et la compensation retenue.
- Fournir les preuves des deux représentations, des refus d'accès, du cache hit et de l'invalidation après mutation.

Validation de l'incrément PDF et images autorisés sont stockés en privé ; Blade et JSON respectent les mêmes Politiques ; le cache reflète chaque mutation validée.

Incrément 13 Déploiement continu du code

Version cible	Résultat observable	Condition de passage
v0.13.0	Le commit validé est déployé automatiquement en staging depuis le code source.	Un commit fusionné est déployé en staging, testé puis restaurable par procédure documentée.

Travail demandé

- Préparer un serveur de staging avec runtime, dépendances et configuration sécurisée.
- Déployer l'archive ou le checkout du commit validé.
- Installer les dépendances de production, exécuter migrations et caches Laravel.
- Ajouter healthcheck, journal de déploiement et procédure de rollback.

Contraintes techniques

- Le déploiement utilise l'identifiant exact du commit.
- Les secrets sont des variables GitLab protégées et masquées.
- Le job de production futur restera manuel et protégé.
- Les migrations sont compatibles avec un déploiement sans perte.

Questions de découverte

56. Pourquoi composer install est utilisé et non composer update.
57. Quelle différence entre artefact de pipeline et workspace du runner.
58. Dans quel ordre exécuter maintenance, migrations, caches et redémarrage.

GitLab imposé

- Branche ci 13 deploy source to staging.
- Commits séparés pour pipeline, script de déploiement et runbook.
- Tag v0.13.0 et Environment GitLab staging.

Docker évolutif

- L'environnement local reste inchangé.
- Documenter le décalage possible entre runtime serveur et runtime conteneurisé.

Pipeline et déploiement

- Pipeline validate, quality, test, package, deploy staging, smoke.
- Déploiement seulement depuis main après succès des Quality Gates.
- Job rollback manuel et smoke test HTTP obligatoire.

DevLog obligatoire

- Insérer le graphe du pipeline, la preuve d'environnement et le temps de reprise.
- Analyser la sécurité des variables et une panne simulée.

Validation de l'incrément Un commit fusionné est déployé en staging, testé puis restaurable par procédure documentée.

Incrément 14 Image immuable et registre GitLab

Version cible	Résultat observable	Condition de passage
v0.14.0	Une image de production testée est publiée puis déployée sans reconstruction.	Le digest testé est celui du conteneur déployé et un rollback vers le digest précédent est démontré.

Travail demandé

- Créer un Dockerfile multi stage avec dépendances de production.
- Exécuter le processus applicatif avec un utilisateur non root.
- Publier l'image dans GitLab Container Registry avec SHA et version.
- Déployer en staging exactement le digest validé puis exécuter un smoke test.

Contraintes techniques

- Aucun secret dans les layers.
- Image minimale, healthcheck et politique de logs documentés.
- latest n'est jamais l'unique référence de déploiement.
- Build once deploy many.

Questions de découverte

59. Quelle différence entre tag d'image mutable et digest immuable.
60. Pourquoi copier composer.lock avant le code améliore le cache.
61. Que doit contenir une image et que doit fournir l'environnement.

GitLab imposé

- Branche ci 14 publish immutable image.
- Commits build docker add production stages puis ci registry publish and deploy digest.
- Tag v0.14.0.

Docker évolutif

- Stages Composer, assets si nécessaires, runtime PHP et serveur.
- Ajouter .dockerignore, utilisateur dédié et permissions minimales.
- Fournir un fichier Compose de production ou un manifeste équivalent.

Pipeline et déploiement

- Jobs image build, image scan si disponible, image push, deploy staging et smoke.
- Promotion en production par job manuel protégé utilisant le même digest.
- Rollback par redéploiement du digest précédent.

DevLog obligatoire

- Comparer déploiement du code et déploiement de l'image.
- Documenter taille, layers, cache, sécurité, digest et rollback.

Validation de l'incrément Le digest testé est celui du conteneur déployé et un rollback vers le digest précédent est démontré.

Incrément 15 Release finale

Version cible	Résultat observable	Condition de passage
v1.0.0	La version stable est documentée, traçable, déployable et démontrable.	La release v1.0.0 est reproductible depuis un clone et toutes les preuves demandées sont disponibles.

Travail demandé

- Exécuter tous les scénarios d'acceptation, y compris workflow, accès, traductions et thèmes.
- Finaliser README, architecture, exploitation, sécurité et décisions.
- Consolider CHANGELOG et publier la GitLab Release.
- Réaliser une démonstration de la fonctionnalité, du pipeline et des deux modes de déploiement.

Contraintes techniques

- Aucun correctif direct sur main.
- Le tag final est créé depuis le commit validé.
- Les preuves sont accessibles sans exposer de secret.

Questions de découverte

62. Quels choix seraient conservés ou modifiés pour une deuxième version.
63. Quelle dette technique demeure et quel risque porte-t-elle.
64. Comment prouver qu'un binaire déployé correspond à une source audité.

GitLab imposé

- Branche release/1.0.0.
- Commits finaux limités à documentation, corrections bloquantes et version.
- Tag annoté v1.0.0 et Release avec notes générées puis relues.

Docker évolutif

- Figuer les images par digest et conserver les versions utiles.
- Documenter sauvegarde, restauration et rotation des journaux.

Pipeline et déploiement

- Pipeline du tag Quality Gates, image, publication, déploiement staging et approbation production.
- Production manuelle et protégée, avec smoke test et rollback.

DevLog obligatoire

- Rédiger une synthèse reliant les seize jalons et les quinze versions intermédiaires de v0.0.0 à v0.14.0.
- Inclure métriques, incidents, arbitrages, dette et plan de suite.

Validation de l'incrément La release v1.0.0 est reproductible depuis un clone et toutes les preuves demandées sont disponibles.

C Stratégie GitLab imposée

Branches

Élément	Convention
Branche stable	main protégée, sans push direct
Branche de travail	feat numéro sujet, fix numéro sujet, ci numéro sujet, docs numéro sujet
Merge Request	Liée à une Issue, petite, relue et pipeline vert
Fusion	Squash autorisé selon convention d'équipe, suppression de la branche
Hotfix	Branche fix depuis main, même Quality Gate, nouveau patch tag

Commits

- Format Conventional Commits type portée description.
- Un commit correspond à une intention testable et réversible.
- Les commits de formatage massif sont séparés des changements fonctionnels.
- Les messages tels que update, fix bug ou travail sont refusés.

Tags et releases

Chaque incrément fusionné reçoit un tag annoté. Les versions 0.x matérialisent la progression pédagogique. La version 1.0.0 correspond au périmètre accepté. Toute correction après release incrémente le patch.

Definition of Done de toute Merge Request

- Issue et critères d'acceptation renseignés.
- Code, tests et documentation synchronisés.
- Pipeline vert et absence de secret.
- DevLog de l'incrément présent.
- Review checklist complétée.
- Tag créé uniquement après fusion sur main.

D Stratégie Docker évolutive

Jalon	Évolution	Intention pédagogique
v0.1.0	Image PHP Composer de développement	Comprendre image, conteneur et bind mount
v0.2.0	Compose app MySQL, réseau, volume, healthcheck	Rendre l'environnement reproductible
v0.3.0	Migrations et modèles	Faire évoluer le schéma sans état manuel
v0.4.0 à v0.10.0	Commandes seed, test et qualité	Aligner poste local et CI
v0.11.0	Configuration MySQL réaliste pour concurrence	Tester le comportement transactionnel
v0.12.0	MinIO, Redis, bucket et matrice Blade JSON	Ajouter stockage objet, cache et représentations configurables
v0.13.0	Le déploiement du code reste volontairement distinct	Observer les limites du runtime serveur
v0.14.0	Dockerfile multi stage de production et Registry	Construire un artefact immuable
v1.0.0	Promotion par digest, rollback et exploitation	Garantir la traçabilité

Règles permanentes

- Aucun secret n'est copié dans l'image.
- Les données sont externes aux conteneurs applicatifs.
- Le processus de production n'est pas root.
- Les images ont un tag SHA et, pour les releases, un tag sémantique.
- Les commandes de développement et de CI utilisent les mêmes scripts de projet.
- Le healthcheck vérifie un état utile et non la simple existence d'un processus.

E Pipeline et déploiement

Stage	Responsabilité	Déclenchement
validate	Composer validate, contrôles de structure et sécurité basique	Toutes branches et Merge Requests
quality	Pint check et analyse statique	Toutes Merge Requests
test	Tests unitaires et Feature avec rapports	Toutes Merge Requests
integration	MySQL puis MinIO et Redis à partir de v0.12.0	Toutes Merge Requests
package	Archive de code puis image OCI selon le jalon	main et tags
deploy staging	Déployer automatiquement l'artefact validé	main
smoke	Vérifier santé et parcours minimal	Après déploiement
deploy production	Promouvoir l'artefact déjà testé	Tag stable et approbation manuelle
rollback	Redéployer la version précédente	Action manuelle protégée

Deux stratégies obligatoires

Déploiement du code

Le pipeline transfère le code ou un artefact source vers le serveur, installe les dépendances de production, prépare les caches Laravel, applique les migrations et redémarre le service. Cette stratégie met en évidence la dépendance au runtime du serveur.

Déploiement par image

Le pipeline construit une image une seule fois, la teste, la publie dans le registre puis déploie son digest. La production promeut exactement le même artefact que le staging. La configuration et les secrets restent externes.

Règle de promotion Une étape de production ne reconstruit jamais l'application. Elle promeut un artefact déjà testé.

F DevLog obligatoire

Emplacement et nom

Un fichier Markdown par version dans docs/devlogs, nommé avec le tag, par exemple v0.8.0.md. Le dossier contient seize DevLogs, de v0.0.0 à v1.0.0.

Structure minimale

Rubrique	Contenu attendu
Objectif	Problème traité et résultat observable
Concepts	Notions rencontrées et définition dans le contexte du projet
Choix	Décision, alternatives, avantages et limites
Implémentation	Éléments essentiels sans recopier tout le code
Problèmes	Symptôme, hypothèses, diagnostic, correction et preuve
Tests	Scénarios, résultats et limites
GitLab	Issue, Merge Request, pipeline, commits et tag
Docker	Évolution de l'environnement et commandes de vérification
Déploiement	Artefact, environnement, smoke test et rollback si applicable
Bilan	Dettes, amélioration et apprentissage personnel

Critères de qualité

- Le DevLog explique les décisions plutôt que de décrire une liste de fichiers.
- Les captures sont accompagnées d'une interprétation.
- Un échec réel et sa résolution sont décrits honnêtement.
- Les liens et identifiants rendent les preuves vérifiables.
- Le vocabulaire technique est utilisé avec précision.

G Scénarios d'acceptation

Numéro	Scénario	Résultat attendu
1	Consulter le catalogue sans compte	Démarches actives visibles ; aucun dossier privé
2	Créer un brouillon pour une démarche active	BROUILLON, propriétaire issu de session, référence unique
3	Soumettre un dossier complet	SOUMIS et historique atomique
4	Soumettre sans pièce obligatoire	DossierIncomplet ; état et historique inchangés
5	Première soumission vers démarche inactive	Refus métier ; brouillon conservé
6	Objet invalide ou statut forgé	Erreurs sans mutation ; champs sensibles ignorés/refusés
7	Citoyen lit le dossier ou la pièce d'un autre	403 ; aucune donnée ou URL temporaire
8	Identifiant inexistant	404 sans donnée sensible
9	Verbe HTTP non autorisé	405 avec méthodes permises
10	Deux décisions concurrentes VALIDE et REJETE	Une seule réussite ; autre conflit de version
11	Déploiement staging par code	Smoke test vert et commit traçable
12	Déploiement staging par image	Digest testé égal au digest exécuté
13	Rollback du code puis de l'image	Version précédente et compatibilité schéma vérifiées
14	PDF et JPEG valides en brouillon	Objets privés et métadonnées créés
15	MIME interdit, taille excessive ou exigence étrangère	Refus sans métadonnée ou objet orphelin
16	Second affichage du catalogue de pièces	Cache hit sans fuite entre dossiers
17	Ajout, retrait ou remplacement après cache	Invalidation ; catalogue mis à jour
18	MEDIA_VIEW_DRIVER=blade	Catalogue autorisé dans une vue Blade
19	MEDIA_VIEW_DRIVER=json	Même catalogue autorisé en JSON
20	Mutation validée puis transaction annulée distincte	Événement unique après commit ; aucun événement sur rollback
21	Agent d'une autre administration	403 ; dossier et pièces inchangés
22	Agent prend SOUMIS en étude	EN_ETUDE, affectation à l'agent et historique
23	Autre agent décide un dossier déjà affecté	403 ; aucune transition

Numéro	Scénario	Résultat attendu
24	Demande de complément sans motif	Refus ; avec motif, COMPLEMENT_REQUIS
25	Citoyen complète puis resoumet	SOUMIS ; affectation conservée ; pièces revérifiées
26	Rejet sans motif puis avec motif	Refus initial ; REJETE terminal avec motif public
27	Annulation en BROUILLON ou SOUMIS	ANNULE ; pièces et historique conservés
28	Annulation en EN_ETUDE ou sortie d'état terminal	TransitionInterdite ; aucun effet
29	Toutes les paires de statuts et les rôles	Seules les transitions de la matrice réussissent
30	MySQL échoue après dépôt dans MinIO	Compensation ou reprise documentée ; pas d'objet abandonné
31	Visiteur puis utilisateur choisit Wolof	fr par défaut ; wo persiste en session puis compte
32	Clé absente en wo ou locale inconnue	Repli fr ; locale refusée ; aucune clé brute visible
33	Système sombre puis bascule manuelle claire	Sombre initial ; choix clair conservé au rechargement
34	Changer thème système avec choix manuel actif	Le choix manuel garde priorité
35	Mobile 360 px et ordinateur dans chaque langue et thème	Pas de débordement ; formulaires, erreurs et focus lisibles
36	Démarche modifiée après soumission	Pièces/conditions enregistrées du dossier restent inchangées
37	Mutation des pièces pendant une soumission	Même verrou ; complétude cohérente, aucune suppression tardive
38	URL temporaire après expiration	Accès impossible ; nouvelle émission après autorisation

Preuve attendue

Chaque scénario est associé à un test automatisé lorsque cela est pertinent. Les scénarios de déploiement sont prouvés par les jobs GitLab, l'état de l'environnement, un smoke test et le DevLog.

H Livrables

- Dépôt GitLab complet avec Issues, Merge Requests, branches supprimées après fusion et tags annotés.
- Code source Laravel, composer.lock et fichiers de configuration non sensibles.
- Migrations, factories, seeders et diagrammes.
- Service MinIO, bucket privé reproductible, cache Redis et tests des pièces justificatives.
- Tests et rapports de qualité.
- Dockerfile de développement puis Dockerfile multi stage de production.
- Fichiers Compose et scripts d'exploitation.
- Pipeline GitLab CI CD fonctionnel.
- Environnement de staging et procédure de production.
- DevLog de chaque incrément.
- README, ARCHITECTURE.md, DEPLOYMENT.md, SECURITY.md et CHANGELOG.md.
- Release GitLab v1.0.0 et image publiée dans le Container Registry.

Documents d'architecture à produire

- Cycle d'une requête Laravel.
- Diagramme de classes et schéma relationnel.
- Décisions d'accès aux données et d'injection.
- Matrices de décision sur le stockage et l'affichage des ressources.
- Diagramme expliquant scopes, Observer, Events, Listeners et sélection Blade JSON par configuration.
- Séquence d'une soumission et d'une décision ; matrice des statuts et mécanisme de concurrence.
- Architecture des conteneurs.
- Graphe du pipeline et flux de promotion des artefacts.

I Évaluation

Axe	Poids	Indicateurs
Fonctionnel et règles métier	25 %	Catalogue, dossiers, pièces, workflow et préférences
Architecture Laravel	20 %	Contrôleurs minces, actions, Politiques et injection justifiée
Données et concurrence	15 %	Migrations, cloisonnement, historique atomique et verrouillage
Tests et qualité	15 %	Matrice, accès, intégrations et Quality Gates
GitLab et versionnement	10 %	Commits, MR, tags, releases et preuves
Docker et CI/CD	10 %	Évolution, image, deux déploiements et rollback
DevLogs et justification	5 %	Décisions, analyse, précision et preuves

Conditions éliminatoires techniques

- Secrets versionnés ou inclus dans une image.
- Transitions ou règles de complétude uniquement dans un contrôleur ou une vue ; absence de cloisonnement des dossiers et pièces.
- Push direct sur main pour contourner une Merge Request.
- Tags ne pointant pas sur les commits annoncés.
- Pipeline ou déploiement simulé sans preuve reproductible.

Vérifications finales du périmètre

- L'authentification et les rôles de base sont requis pour protéger les citoyens et agents, et sont validés avant v1.0.0.
- La représentation JSON de l'incrément 12 reste un exercice interne de substitution ; aucune API publique étendue n'est demandée.
- Le workflow, Français/Wolof et le mode clair/sombre sont les trois évolutions fonctionnelles retenues.
- Paiements, QR Code, identité numérique avancée, génération de documents officiels, notifications et tableaux de bord analytiques ne font pas partie de cette release.
- Toute extension fonctionnelle nécessite un cadrage et une validation distincts après le parcours.
- La release est une livraison pédagogique démontrée ; elle ne constitue pas un raccordement aux systèmes de l'État.

J Questions de synthèse

65. Décrire le trajet complet d'une soumission de dossier jusqu'à son historique et sa réponse Blade.
66. Distinguer validation HTTP, invariant métier et contrainte de base.
67. Expliquer pourquoi le contrôleur ne doit pas décider des transitions du dossier.
68. Justifier l'usage ou l'absence d'un Repository avec Eloquent.
69. Comparer disque local privé, BLOB MySQL et stockage objet MinIO pour les pièces justificatives.
70. Expliquer le cycle du cache du catalogue de pièces, le TTL, l'invalidation et la protection entre citoyens.
71. Distinguer scope local, scope global, Observer, Event et Listener sur des exemples du projet.
72. Expliquer comment MEDIA_VIEW_DRIVER traverse .env et config media.php pour sélectionner Blade ou JSON.
73. Expliquer comment le conteneur Laravel applique l'inversion de contrôle.
74. Démontrer sur quatre cas la matrice des transitions et les conséquences d'une version attendue périmée.
75. Expliquer pourquoi une transaction seule peut être insuffisante.
76. Comparer test unitaire, Feature test et test d'intégration MySQL.
77. Comparer déploiement du code et déploiement d'une image.
78. Prouver la correspondance entre commit, tag, image et version déployée.
79. Présenter une dette technique, les tests qui la bornent et le prochain incrément qui la réduirait.
80. Expliquer comment les Politiques et les requêtes filtrées protègent ensemble un dossier et ses pièces.
81. Justifier l'écriture de l'historique dans la transaction et les réactions secondaires après commit.
82. Expliquer la priorité entre langue du compte, session et français de repli.
83. Distinguer code technique de statut et libellé français ou Wolof.
84. Expliquer la priorité entre thème manuel et préférence système, puis tester le rechargement.
85. Décrire la compensation d'un échec MySQL après dépôt MinIO et le maintien d'une seule pièce active.

Résultat attendu Un étudiant ayant terminé le parcours doit être capable de reprendre une application Laravel existante, de localiser chaque responsabilité, d'ajouter un cas d'usage testé et de le livrer par une chaîne automatisée sans dégrader la traçabilité.

Références techniques

Document pédagogique de référence : projet_laravel_reservation_salles_demarche_incrementale.docx. Le parcours conserve ses jalons, sa progression technique, ses livrables et sa grille d'évaluation adaptés au domaine administratif.

Documentation officielle consultée le 29 septembre 2026. Les versions de PHP, Laravel, MySQL, des outils de qualité et des images sont figées au jalon 1 et dans les fichiers de verrouillage ; leur compatibilité est vérifiée en CI.

Laravel versions et compatibilité PHP <https://laravel.com/docs/12.x/releases>

Laravel localisation <https://laravel.com/docs/12.x/localization>

Laravel Eloquent et Observers <https://laravel.com/docs/12.x/eloquent>

GitLab déploiements et environnements <https://docs.gitlab.com/ci/environments/deployments/>